Incentive-Aware Multi-Objective Matching for Peer-to-Peer Commuter Carpooling

Krunal Savaj [1]

Abstract

1. Introduction

Urban commuting accounts for a significant share of daily vehicle miles traveled in the United States, yet the average private car carries only 1.08 occupants during peak hours (Santos et al., 2011). The math is simple: if even a fraction of those solo commuters shared rides with someone heading the same direction, the impact on congestion, cost, and emissions would be substantial, and it requires zero new infrastructure.

Platforms like Uber and Lyft already offer pooled rides, but their matching algorithms are built around professional drivers who accept any profitable trip. Peer-to-peer (P2P) carpooling between regular commuters is a fundamentally different problem. These are people with fixed routes, fixed schedules, and limited tolerance for detours. A commuter who drives from the Bronx to Midtown every morning at 8:15 AM is not going to accept a 6 km detour to pick up a stranger unless the economics make sense.

The Learning Problem. We frame this as a supervised prediction task followed by combinatorial optimization. Given a set of n drivers $\mathcal{D} = \{d_1, \ldots, d_n\}$ and m riders $\mathcal{R} = \{r_1, \ldots, r_m\}$, each defined by an origin–destination coordinate pair, pickup time, and fare, we construct a candidate pair (d_i, r_j) whenever the geodesic distance between their origins falls within a configurable spatial threshold δ_{max}.

For each candidate pair, we engineer a feature vector $\mathbf{x}_{ij} \in \mathbb{R}^5$ capturing route overlap, detour cost, temporal compatibility, and economic incentives. The prediction task has two objectives:

1. **Regression:** Predict the additional detour $\hat{y}_{ij}^{\text{det}} \in \mathbb{R}^+$ the driver would incur.
2. **Classification:** Predict the probability $\hat{y}_{ij}^{\text{acc}} \in [0, 1]$ that the driver accepts the match.

These dual predictions are fused into a composite match score $s_{ij} = \hat{y}_{ij}^{\text{acc}} - \lambda \cdot \hat{y}_{ij}^{\text{det}}$, where λ controls the detour penalty. At inference time, the scores serve as edge weights in a bipartite graph $G = (\mathcal{D}, \mathcal{R}, E)$, and we solve for the maximum-weight matching to produce globally optimal one-to-one assignments.

Why This is Hard. This problem is hard for a few reasons. No public P2P carpooling dataset exists. Real matching data is proprietary (Uber, Waze), so we have to construct a simulation pipeline that synthesizes labeled training pairs from 3.5 million single-passenger NYC taxi trips. The acceptance decision itself is inherently multi-factor: a driver's willingness depends on the interaction between detour severity, fare compensation, timing, and the detour-to-trip-length ratio, not any single threshold. And even with good pairwise predictions, greedy assignment does not guarantee a good global outcome. A locally attractive match can block a better pairing downstream.

Our Approach. Our pipeline has three stages: (1) a data synthesis engine that generates geographically realistic driver–rider pairs from public taxi records using corridor-aware coordinate projection and multi-factor stochastic labeling, (2) a dual-objective XGBoost model (Chen & Guestrin, 2016) that simultaneously predicts detour cost and acceptance probability, and (3) a bipartite matching layer (Kuhn, 1955) that selects globally optimal assignments using the predicted scores as edge weights. This architecture follows the learning-augmented matching paradigm (Bjorck et al., 2021), which provably outperforms either pure heuristic matching or pure ML prediction alone.

2. Related Work

Ride-sharing optimization has been studied from several angles: shareability analysis, matching algorithms, trajectory-based routing, and multi-objective formulations. We review

[1]Khoury College of Computer Sciences, Northeastern University, Portland, ME. Correspondence to: Krunal Savaj <savaj.k@northeastern.edu>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

each below.

Shareability Networks and Fleet Optimization. The foundational work of Santi et al. (2014) introduced *shareability networks*, modeling every NYC taxi trip as a node and connecting shareable trips with edges. By solving bipartite matching on this graph, they showed that ride-sharing could reduce the taxi fleet by 40% with minimal passenger delay. Their approach relies entirely on fixed spatiotemporal thresholds to determine shareability: if two trips can be combined within a delay budget, they are connected. While powerful for fleet-level analysis, this binary threshold ignores the nuance of *individual* willingness: a 3 km detour is acceptable on a 30 km highway commute but painful on a 5 km city trip. Our work builds directly on their graph formulation but replaces the fixed threshold with a learned acceptance model that captures these non-linear trade-offs.

Matching with Learned Predictions. Bjorck et al. (2021) formalized the concept of *learning-augmented matching*, proving that using ML predictions as edge weights in bipartite optimization provably outperforms both pure heuristic matching and pure ML prediction used in isolation. Their theoretical guarantees show that even when predictions are imperfect, the matching quality degrades gracefully (never worse than $2\times$ optimal). They validate on taxi-rider matching with uncertain trip duration. Our pipeline directly instantiates this paradigm: XGBoost generates match scores, and maximum-weight bipartite matching (Kuhn, 1955) selects globally optimal assignments at inference time.

Spatial and Temporal Pre-filtering. Efficient candidate generation is critical at scale. Brute-force comparison of all driver–rider pairs is $O(n^2)$ and infeasible for large networks. Anagnostopoulos & Ramson (2025) address this with a three-dimensional matching framework incorporating spatial, temporal, and social behavior similarity, tested on Athens taxi data. Their social dimension (modeling whether passengers have compatible travel habits) is particularly relevant for P2P settings with repeat commuters. We adopt a similar spatial and temporal pre-filtering strategy, using Ball-Tree indexing with the Haversine metric (Omohundro, 1989) for $O(\log n)$ neighbor queries within configurable geodesic radii, combined with rolling temporal batch windows.

Trajectory-Based Routing. Hsieh (2017) proposed matching based on full GPS trajectories rather than origin–destination points, integrating Google Maps API for real road-network distances. This captures critical routing details that straight-line (geodesic) distance misses: a rider 3 km away across a river with no bridge requires a 20-minute detour, while one 3 km down the highway requires 30 seconds. Our current implementation uses geodesic distance via `geopy` (GeoPy Contributors, 2023), which is a known limitation. However, for commuter corridors where routes are predominantly highway-based, the geodesic approximation is reasonable, and we discuss OSRM integration as future work.

Gradient Boosted Trees for Tabular Geospatial Data. For structured, tabular prediction tasks, XGBoost (Chen & Guestrin, 2016) tends to outperform deep learning when the feature space has moderate dimensionality and mixed types (Grinsztajn et al., 2022). Several recent studies have applied XGBoost specifically to ride-sharing contexts, including dynamic rider grouping in NYC based on route and time similarity (Ma & Li, 2023). These works validate that gradient-boosted trees are well-suited to our feature space (distances, ratios, fares, timestamps) and motivate our choice of XGBoost over neural architectures.

Multi-Objective Optimization Approaches. An alternative paradigm treats ride-matching as a multi-objective optimization problem. Wang et al. (2023) apply NSGA-II (a Pareto-based evolutionary algorithm) to minimize both driver detour time and passenger travel time subject to time-window and vehicle capacity constraints. While evolutionary methods can explore the Pareto frontier directly, they are computationally expensive and do not produce a trained model that generalizes to unseen pairs. Our approach separates prediction (learning the detour–acceptance relationship) from optimization (bipartite matching), which allows the predictive model to generalize across regions and the matching layer to be swapped independently.

How Our Work Differs. While shareability networks (Santi et al., 2014) and learning-augmented matching (Bjorck et al., 2021) have separately addressed ride-pooling optimization, no prior work combines all of the following: (1) a reproducible data synthesis pipeline that generates labeled P2P training pairs from public single-passenger taxi records using corridor-aware coordinate projection, (2) a dual-objective gradient-boosted model that simultaneously predicts detour cost and acceptance probability, (3) multi-factor stochastic labeling with Gumbel noise to simulate realistic human decision-making, and (4) a bipartite matching layer that uses the predicted scores for globally optimal assignments. As a baseline, we use logistic regression (Cox, 1958) via scikit-learn (Pedregosa et al., 2011), which cannot capture these interaction effects, motivating the non-linear XGBoost model. Our features are dimensionless ratios (e.g., overlap ratio) rather than raw coordinates, so the trained model transfers across cities without retraining.

Table 1. Feature definitions for each candidate pair.

Feature	Type	Description
driver_orig_dist	float	Driver's solo trip (km)
overlap_ratio	float	Route overlap $\in [0, 1]$
time_diff_mins	float	Pickup time gap (min)
driver_approx_fare	float	Driver fare ($)
rider_approx_fare	float	Rider fare ($)
detour_distance_km	float	Regression target (km)
label	binary	Accept/reject (1/0)

3. Data

Source and Construction. No public P2P carpooling dataset exists, so we construct one from the NYC Taxi & Limousine Commission (TLC) Yellow Taxi trip records for August 2025 (3.5 million single-passenger trips). Since the TLC replaced exact GPS coordinates with integer zone IDs (`PULocationID`) in 2016, we synthesize pickup coordinates via Zonal Bounding Box Distribution over official shapefile polygons with point-in-polygon rejection sampling, and synthesize dropoff coordinates via forward-azimuth spherical projection scaled to the recorded `trip_distance`. Details are in our public codebase (`src/data/loader.py`).

From these trips, we simulate a P2P marketplace: 50% of commute-window (7–10 AM) trips are randomly assigned as drivers, 50% as riders. A BallTree spatial index (Haversine metric, 5 km radius) identifies nearby driver–rider candidates within 5-minute temporal batches. For each feasible pair, we compute 5 engineered features and assign a stochastic acceptance label.

Features. Table 1 describes the 5 input features and 2 prediction targets.

Dataset Size and Label Balance. The simulation produces 750 labeled pairs (dev mode, 5% sample). In production mode (full dataset, 25K driver limit), this scales to $\sim$10K+ pairs. The label split is 43.3% accepted / 56.7% rejected, which is roughly balanced and consistent with our 0.45 acceptance threshold plus Gumbel noise. No feature has missing values; all are computed deterministically from the synthesis pipeline.

Train/Test Split. We use a single 80/20 stratified split (`random_state=42`) shared across both the regressor and classifier, ensuring metrics are evaluated on identical test rows. We validate stability via 5-fold cross-validation on the training partition.

Exploratory Analysis. Figure 1 shows feature distributions split by label. Accepted pairs cluster at lower detour distances and higher overlap ratios, confirming that the stochastic labeling produces realistic patterns. The model has to actually learn the boundary, not just memorize a trivial threshold.

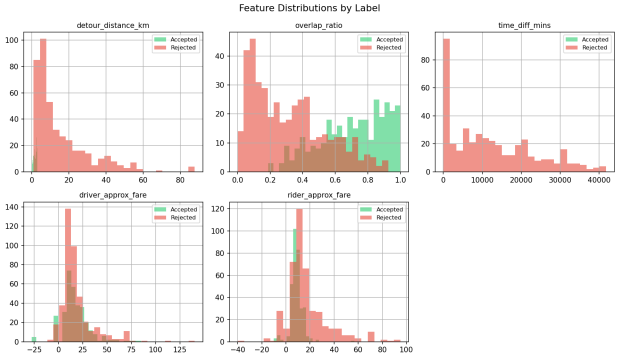


Figure 1. Feature distributions by acceptance label. Accepted pairs (green) show lower detour and higher overlap than rejected pairs (red).

4. Methods

4.1. Evaluation Metrics

We evaluate two prediction tasks with complementary metrics:

Regression (detour prediction): Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). MAE gives the average prediction error in km; RMSE penalizes large errors more heavily, which matters because a 10 km misprediction is much worse than two 5 km errors.

Classification (accept/reject): Accuracy, Precision, Recall, F1-score, and ROC-AUC. We prioritize F1 over raw accuracy because the classes are close to balanced (43/57 split) but not perfectly so. ROC-AUC measures ranking quality across all thresholds, which matters here because the bipartite matcher uses continuous probability scores, not hard 0/1 predictions.

4.2. Baseline Models

Rule-Based Baseline. A zero-learning heuristic: accept any pair where detour $\leq \delta_{\max}$ and time difference ≤ 15 min. This is what a naive app would do. Thresholds are read from the same config file that generated the training data, ensuring a fair comparison.

Logistic Regression. A linear classifier trained via scikit-learn with balanced class weights. Given feature vector $\mathbf{x}$, it models:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) \tag{1}$$

where σ is the sigmoid function. Features are standardized via `StandardScaler` (fit on training data only). Logistic

regression cannot capture interaction effects (e.g., a 2.5 km detour is fine when the fare is \$30 but not when it's \$8), which motivates the non-linear model.

4.3. Advanced Model: Dual-Objective XGBoost

Our advanced model is an ensemble of gradient-boosted decision trees (Chen & Guestrin, 2016) with two heads:

Regressor minimizes squared error on detour prediction:

$$\mathcal{L}_{\text{reg}} = \sum_i (y_i^{\text{det}} - \hat{y}_i^{\text{det}})^2 + \Omega(f) \tag{2}$$

Classifier minimizes log loss on acceptance prediction:

$$\mathcal{L}_{\text{clf}} = -\sum_i \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right] + \Omega(f) \tag{3}$$

where $\Omega(f) = \gamma T + \frac{1}{2}\lambda \|\mathbf{w}\|^2$ is the XGBoost regularization term (T = number of leaves). Both models share the same 5 input features but predict different targets.

At inference, their outputs are fused into a composite match score (as defined in Section 1), which serves as edge weights in the bipartite matching layer.

4.4. Training Procedure

Hyperparameter tuning. We run exhaustive grid search over `n_estimators` $\in \{50, 100, 200\}$, `max_depth` $\in \{3, 5, 7\}$, and `learning_rate` $\in \{0.05, 0.1, 0.2\}$ (27 combinations) with 5-fold cross-validation on the training partition. The best configuration is selected by accuracy.

Validation. After selecting hyperparameters, we run 5-fold CV on the training set with the best parameters to confirm stability. We report mean $\pm$ standard deviation across folds.

Final evaluation. The tuned model is retrained on the full 80% training set and evaluated on the held-out 20% test set. All reported metrics come from this test set, which was never seen during training or hyperparameter selection.

Hardware. All experiments run on a single-core CPU (Apple M-series). Grid search completes in under 30 seconds on the 750-pair dev dataset; production runs ($\sim$10K pairs) take approximately 5 minutes.

5. Results

6. Discussion

6.1. Limitations

6.2. Ethical Considerations

7. Conclusion

References

Anagnostopoulos, T. and Ramson, S. R. J. A vehicle ride-sharing algorithm assessing passenger satisfaction according to spatial, temporal, and social behavior context. *Future Transportation*, 5(2), 2025. doi: 10.3390/futuretransp5020023.

Bjorck, J., Shi, Q., Brown-Lima, C., Dean, J., Fuller, A. K., and Gomes, C. P. Learning augmented methods for matching: Improving invasive species management and urban mobility. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pp. 15520–15521, 2021.

Chen, T. and Guestrin, C. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. ACM, 2016. doi: 10.1145/2939672.2939785.

Cox, D. R. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B*, 20(2): 215–242, 1958.

GeoPy Contributors. GeoPy: Geocoding library for Python. `https://geopy.readthedocs.io/`, 2023. Accessed: 2025-03-25.

Grinsztajn, L., Oyallon, E., and Varoquaux, G. Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems*, 35:507–520, 2022.

Hsieh, F.-S. Car pooling based on trajectories of drivers and requirements of passengers. In *IEEE 31st International Conference on Advanced Information Networking and Applications (AINA)*, pp. 972–979. IEEE, 2017. doi: 10.1109/AINA.2017.101.

Kuhn, H. W. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2): 83–97, 1955. doi: 10.1002/nav.3800020109.

Ma, J. and Li, W. XGBoost-based dynamic ride-sharing matching model for NYC taxi data. *ResearchGate Preprint*, 2023.

Omohundro, S. M. Five balltree construction algorithms. Technical Report TR-89-063, International Computer Science Institute, 1989.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Santi, P., Resta, G., Szell, M., Sobolevsky, S., Strogatz, S. H., and Ratti, C. Quantifying the benefits of vehicle pooling with shareability networks. *Proceedings of the National Academy of Sciences*, 111(37):13290–13294, 2014. doi: 10.1073/pnas.1403657111.

Santos, A., McGuckin, N., Nakamoto, H. Y., Gray, D., and Liss, S. Summary of travel trends: 2009 national household travel survey. Technical Report FHWA-PL-11-022, U.S. Department of Transportation, Federal Highway Administration, 2011.

Wang, Y., Liu, X., and Zhang, H. Multi-objective optimization for commuter carpooling. *MDPI Sustainability*, 15 (8), 2023.